

Bloom Filters en Cassandra

Una estructura probabilística para evitar búsquedas innecesarias en SSTables

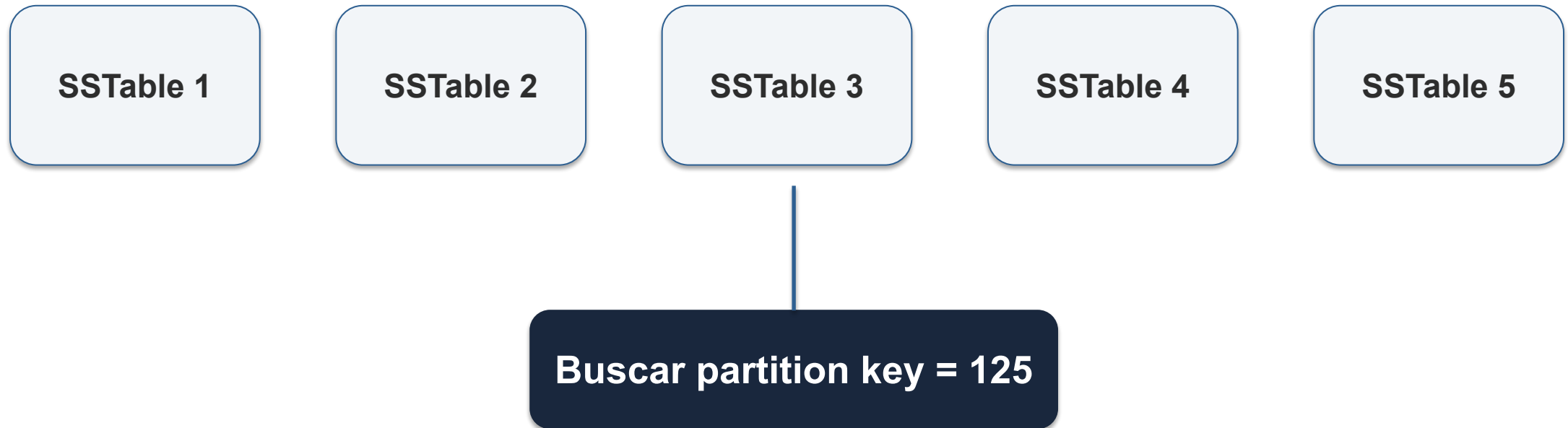
“Puede decir con certeza NO está; si dice puede estar, hay que comprobar.”

NO
→ **definitivo**

TAL VEZ
→ **comprobar**

1. El problema

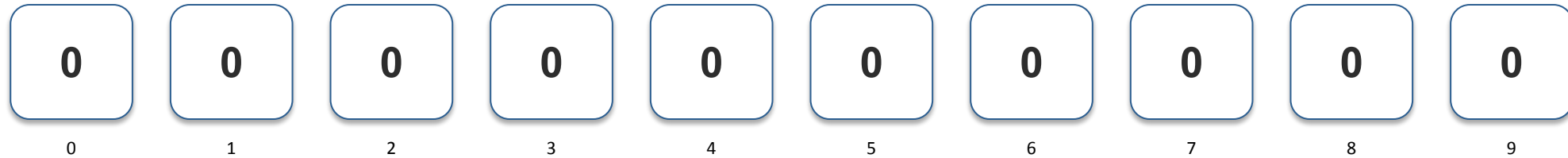
Una lectura puede tener que considerar varias SSTables



Pregunta: ¿cómo descartar rápidamente SSTables donde la clave seguro no existe?

2. ¿Qué contiene un Bloom Filter?

Un vector de bits + varias funciones hash



h1(clave)

h2(clave)

h3(clave)

Cada hash selecciona una posición del vector.

3. Insertar una clave

Ejemplo: Juan → posiciones 2, 5 y 8



$h1(\text{Juan})=2$ $h2(\text{Juan})=5$
 $h3(\text{Juan})=8$

Al insertar, esas posiciones pasan a 1.

4. Consultar: ¿está Pedro?

Supongamos $h(\text{Pedro}) \rightarrow$ posiciones 2, 4 y 8



Aparece un 0 → Pedro DEFINITIVAMENTE NO fue agregado

No hay falsos negativos.

5. ¿Y si todas las posiciones son 1?

El Bloom Filter sólo puede responder: “posiblemente está”

Bloom Filter: PUEDE ESTAR

**Luego hay que comprobar la
SSTable**

NO está

No está



NO está

Sí está

✗ imposible

Puede estar

Sí está



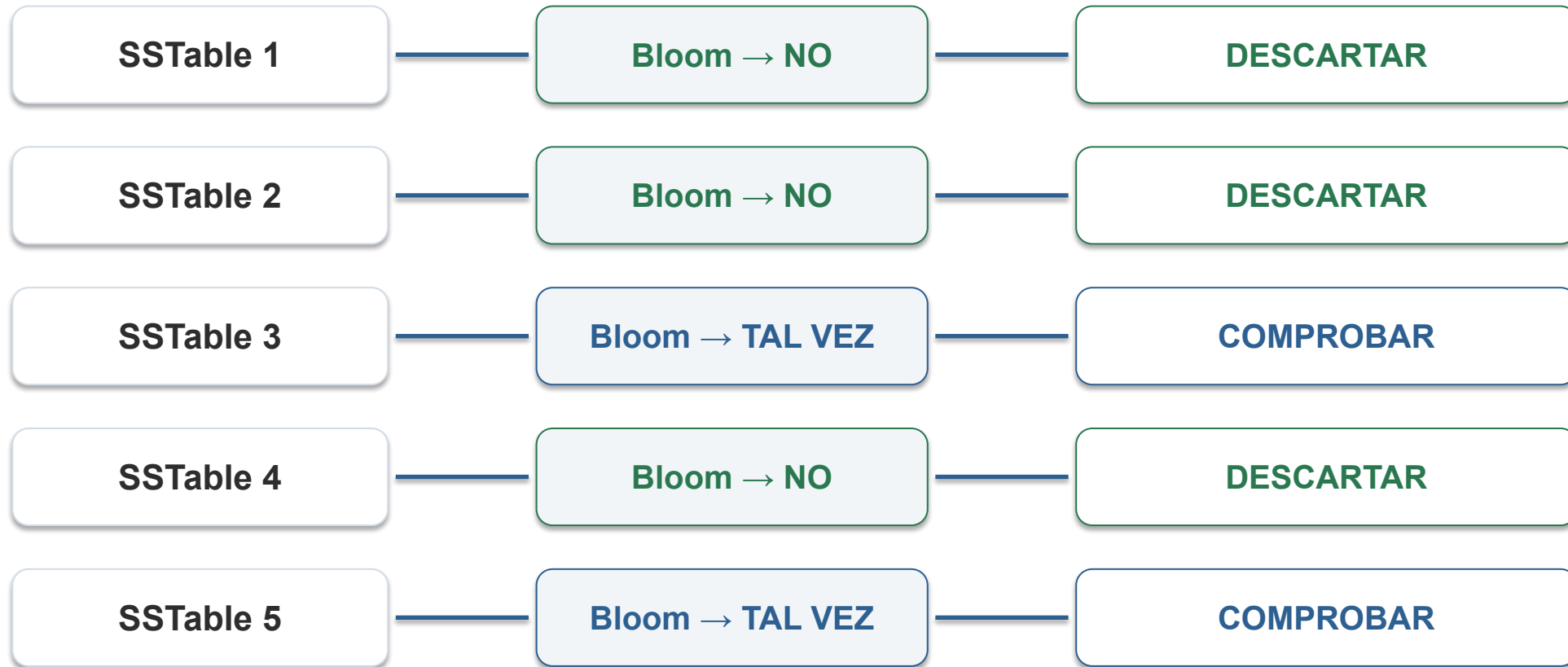
Puede estar

No está

✓ falso positivo

6. ¿Cómo lo usa Cassandra?

El Bloom Filter no encuentra el dato: descarta dónde no vale la pena buscar



Más memoria para el filtro → menos falsos positivos → menos búsquedas innecesarias

7. Para cerrar la explicación

Tres ideas que deberían llevarse

1 · Un Bloom Filter es probabilístico y muy compacto.

2 · “NO” es seguro; “puede estar” requiere comprobación.

3 · Cassandra lo usa para evitar trabajo innecesario sobre SSTables.

Pregunta: ¿por qué es aceptable tener falsos positivos, pero no falsos negativos?